

Machine Project II (GPU)

CS 521 — Machine Learning and Compilers
Fall Semester 2026

Due: 11:59pm (Urbana, IL Time Zone) Thursday October 8.
--

Goal

In this project, you will implement kernel-level optimizations *manually* on the GPU, using several different compilers and programming models. In particular, we will explore optimizations for the 2D convolution kernels. The goal is to assimilate yourself with kernel-level optimizations done for typical deep learning operators and to appreciate the complexities in optimizing them using different compiler tools and languages. We will optimize the 2D convolution kernel on the NVIDIA GPU; a companion project optimizes the same kernel on the Amazon Trainium platform. We will release the Trainium part soon, after we set the environment in collaboration with Amazon AWS engineers.

Kernel Descriptions

2D Convolution (conv2d) For input matrix A of size $N \times M$ and filter matrix W of size $K_1 \times K_2$, the elements of the convolved output matrix C can be computed as,

$$C_{n,m} = \sum_{k_1=0}^{K_1-1} \sum_{k_2=0}^{K_2-1} W_{k_1,k_2} * B_{n+k_1,m+k_2}$$

This definition does not cover padding, dilation or strides that we mentioned in the class.

Hardware Environments

GPU: Google Colab You will develop and execute the GPU code on Google Colab.

- Start a new notebook at <https://colab.research.google.com>. Before running anything, switch its runtime to a GPU: *Runtime* → *Change runtime type* → *Hardware accelerator* → *T4 GPU*. The free tier's T4 is what this assignment is designed and graded on.
- Get the code into the session and run it from a notebook cell:

```
!nvidia-smi
!git clone https://github.com/<your-username>/mp2.git
%cd mp2/gpu
!python3 myconv.py
```

Instead of cloning, you can upload the `gpu/` files by hand: open the file browser in the left sidebar and drag them in.

- Colab's filesystem is erased when the runtime shuts down, and idle sessions are disconnected. Anything you want to keep, such as edited sources and profiler traces, must be pushed back to your own repository or downloaded (right-click the file in the sidebar → *Download*) before you close the session.

Starter Code

The starter code for this MP can be found in <https://github.com/CS521-ml-compilers/mp2>. Code for each hardware environment (GPU/NPU) is in its own directory; this project uses the GPU directory only. You will run the GPU code on Google Colab.

You will run Python code as usual, with the following. Refer to Guidelines on Tools section to complete your GPU environment setup.

```
python3 myconv.py
python3 myconv_inductor.py
python3 myconv_jax.py
python3 myconv_interface.py
```

Hand-in Instructions

You will hand-in both the code and a report on certain experimental results as detailed under each section. Detailed instructions on how to submit the code and the report will be given closer to the submission deadline.

Guidelines on Tools

- Colab already provides Python, PyTorch with CUDA, and JAX with CUDA support. You do not need to install PyTorch, cuDNN, or JAX yourself, and you should not `pip install` a different version of them, since doing so tends to break the CUDA libraries that ship with the runtime. If you want to read about the JAX install for other machines, see the JAX Installation Guide.
- Confirm that JAX actually sees the GPU before you trust any JAX number: `jax.default_backend()` must print `gpu`. If it prints `cpu`, your runtime is not a GPU runtime.
- To profile Python code, refer to Pytorch Profiler documentation. Here are some example usecases that wrap compute kernels and monitor memory footprint. Make sure to always initialize your profiler with below code to capture GPU activity.

```
activities=[ProfilerActivity.CUDA]
```

- You should export Pytorch profiler traces as `.json` or `.json.gz` files to load them into Perfetto UI. Here is the documentation for using Perfetto.

Part 1: Tensor Programming and Einsum Notation (15 points) [No AI help allowed]

Einsum notation is a powerful programming construct that can represent many tensor computations. For the following tensor computations, if they are representable as einsum give its representation and if not mention why. The semantics of the tensor operators follow the usual PyTorch semantics. You are allowed to look up the semantics online.

Use $i, j, k, l, m, \dots$ as index variables. Use the notation we discussed in the class. Each part is worth 3 points. This is the first theory assignment in this class; you only need to answer it in the report (no code necessary).

1. `O = Reduce(A, dim=i)`; A is a matrix.
2. `O = Convolution(A, B)`; A and B are matrices. A is the input and B is the kernel.
3. `O = BatchedMatMul(A, B)`; A and B are 3-d tensors where the batch dimension is the outermost (i).
4. `O = ElementWise(Relu, A)`; A is a 3-d tensor.
5. `O = ElementWise(Mul, A, B)`; A and B are 3-d tensors. This is also known as the hadamard product.

Part 2: Tensor Computational Graphs and Optimizations (15 points) [No AI help allowed]

Consider the following PyTorch program. All tensors are of rank 2.

```
O1 = torch.matmul(A, B)
O2 = torch.matmul(C, O1)
O3 = torch.matmul(C, D)
```

We will be using the following two rewrite rules taken from the TASO paper Figure 2.

```
R1: matmul(A, matmul(B,C)) => matmul(matmul(A,B), C)
R2: [matmul(A,B); matmul(A,C)] => split(matmul(A, concat(B,C)))
```

1. (2 points) Draw the Tensor Computational Graph (TCG) for the above code snippet. You can assume that the same operators are available at the IR level without the torch prefix. Also, note that the direction of edges should be from producer to consumer operator. Use circles for tensor inputs and rectangles for operators. Operand position is captured from left to right. The outputs are O2 and O3.
2. (4 points) Draw the TCG after applying R1 and R2 in that order to the TCG of (a).
3. (3 points) Draw the TCG after applying R2 and R1 in that order to the TCG of (a).
4. (2 points) Are there any differences in TCGs for (b) and (c)? Explain your answer
5. (4 points) When would the application order of (b) be more beneficial than (c)? You can use the sizes of dimensions to argue your answer (Hint: focus on the R1 rule).

Part 3: Writing, Compiling, and Profiling the Convolution Kernel on GPU (15 points for im2col + 25 points for code + 20 points report)

(15 pts) In this part, you will implement a tiled convolution kernel in PyTorch by manually implementing the *im2col* operator (discussed in Lecture 3). You may NOT use PyTorch's fold/unfold built-in calls to perform *im2col* operation. Instead, you are expected to use combinations of built-in PyTorch operations like reshape, permute, stack to implement im2col from scratch. You must run this convolution kernel on the GPU.

(25 pts) After having the convolution operation completed in PyTorch, you will compile and run it using the following frameworks: PyTorch interpreter, PyTorch Inductor compiler, and the JAX compiler with JIT compiling enabled. You may refer to JAX documentation to adapt your convolution code in PyTorch to JAX.

The learning goal of this part of the MP is to identify various compiler optimizations, as well as their performance impacts on your GPU kernel. To do so, you will first export execution traces from all three versions via Pytorch Profiler and visualize them in the Perfetto UI. Perfetto provides an intuitive interface for exploring convolution kernels and understanding compiler optimizations.

We provide you with the skeleton of the model class for your convolution solution, as well as algorithmic hints for implementing the im2col operation and convolution. Additionally, we included usage examples of JAX/Inductor compilation in the source files. Make sure to closely follow the comments in the source code.

Additionally, you will implement a tiled convolution kernel in CUDA. You will explore tiled convolution by leveraging shared memory buffers, analogous to PSUM buffers used in NPU architectures (which will be useful when implementing the Trainium code). We have integrated this CUDA kernel with PyTorch via its C++ extension API, allowing it to be invoked just like a standard PyTorch call. This integration lets you verify the correctness of your kernel by comparing its output with PyTorch's built-in conv2d operator. In your report, similarly include the performance of this part in the plots and discuss CUDA kernel performance against above three versions.

Note: You may use your previous GEMM codes from MP1 if you believe that it would help you with implementing convolution.

(20 pts) In your report, discuss kernel execution times on the GPU and host compilation times for the four convolution variations. Provide a plot for walltimes of your kernel you observe in the traces, while varying input and filter sizes. Justify if the observed results are expected based on the compiler you have used by mentioning its low-level optimizations (for Inductor and JAX). You can find such optimizations as high level function calls on the host side in Perfetto UI. Summarize how performance varies across compilers for different input shapes in a single paragraph.